POSTECH FIAP — ARQUITETURA E DESENVOLVIMENTO JAVA

Tech Challenge Fase 1

Documentação Técnica do Projeto — Restaurante Gestão de Usuários

FRAMEWORK	LINGUAGEM	BANCO DE DADOS
Spring Boot 3.2.5	**Java 17**	**PostgreSQL 16**
MIGRATIONS	CONTAINERIZAÇÃO	DOCUMENTAÇÃO API
Flyway	**Docker + Compose**	**Swagger / OpenAPI 3**

Gerado automaticamente • Abril 2026

Sumário

1. Visão Geral do Projeto
Objetivo, tecnologias e entregáveis — 3

2. Arquitetura em Camadas
Diagrama, responsabilidades e fluxo de dados — 4

3. Camada de Domínio — Model
Entidades JPA, Address embeddable, UserType enum — 5

4. Repositório — UserRepository
Spring Data JPA, queries derivadas — 7

5. Migrations com Flyway
Estratégia de versionamento e script SQL — 8

6. Configurações da Aplicação
application.yml, perfil de teste, variáveis de ambiente — 9

7. Segurança — SecurityConfig
BCryptPasswordEncoder, filtro HTTP — 10

8. Swagger / OpenAPI
OpenApiConfig e endpoint de documentação — 11

9. Infraestrutura Docker
Dockerfile multi-stage e docker-compose.yml — 12

10. Dependências (pom.xml)
Detalhamento de cada dependência Maven — 13

11. O que falta implementar
Pendências identificadas para completar o projeto — 14

1 Visão Geral do Projeto

O projeto é um **backend REST em Spring Boot** para gestão de usuários de um sistema de restaurantes, desenvolvido como entregável do Tech Challenge Fase 1 da POSTECH FIAP. O sistema suporta dois tipos de usuário — **Dono de Restaurante (OWNER)** e **Cliente (CUSTOMER)** — e expõe operações de CRUD, troca de senha e validação de login.

Objetivo

- Cadastrar e gerenciar usuários (CRUD completo)
- Troca de senha com validação BCrypt
- Validação de login (autenticação simples)
- API documentada via Swagger/OpenAPI 3
- Ambiente 100% containerizado com Docker

Entregáveis

- Código-fonte no GitHub
- Docker Compose funcional
- Collection Postman (pendente)
- Documentação Swagger em `/swagger-ui.html`
- README detalhado

Stack de Tecnologias

Categoria	Tecnologia	Versão	Finalidade
Framework	Spring Boot	3.2.5	Base da aplicação REST
Linguagem	Java	17 (LTS)	Linguagem principal
Persistência	Spring Data JPA / Hibernate	—	Mapeamento objeto-relacional
Banco (prod)	PostgreSQL	16	Banco relacional de produção
Banco (teste)	H2	—	Banco in-memory para testes
Migrations	Flyway	—	Controle de versão do schema SQL
Segurança	Spring Security	—	BCryptPasswordEncoder

Categoria	Tecnologia	Versão	Finalidade
Docs API	Springdoc OpenAPI	2.5.0	Swagger UI automático
Boilerplate	Lombok	—	Redução de código repetitivo
Container	Docker + Compose	—	Empacotamento e orquestração

2 Arquitetura em Camadas

A aplicação segue o padrão **arquitetura em camadas (Layered Architecture)**, onde cada camada tem uma responsabilidade única e bem definida. A comunicação entre camadas é sempre de cima para baixo — a camada superior depende da inferior, nunca o contrário.

HTTP Client	Postman / Browser / Frontend — faz requisições HTTP (GET, POST, PUT, DELETE, PATCH)

↓ JSON sobre HTTP ↓

Controller	Recebe requisições, valida @Valid nos DTOs, delega para o Service, retorna ResponseEntity

↓ chama ↓

Service	Contém toda a lógica de negócio: unicidade, BCrypt, regras de atualização, exceções customizadas

↓ chama ↓

Repository	Interface Spring Data JPA — abstrai acesso ao banco. Queries derivadas por nome de método

↓ SQL gerado pelo Hibernate ↓

Model (Entity)	Entidades JPA mapeadas para tabelas SQL. User @Entity + Address @Embeddable

↓ JDBC ↓

PostgreSQL	Banco de dados relacional — schema criado e versionado pelo Flyway

Estrutura de Pacotes

```
br.com.fiap.restaurante ├── config/ ← SecurityConfig, OpenApiConfig ├── controller/ ← UserController (a implementar) ├── dto/ ← Request / Response DTOs (a implementar) ├── exception/ ← Exceções customizadas + GlobalExceptionHandler (a implementar) ├── model/ ← User.java, Address.java, UserType.java ✔ ├── repository/ ← UserRepository.java ✔ └── service/ ← UserService (a implementar)
```

Papel de cada camada

Pacote	Responsabilidade	Status
`config`	Beans de configuração global (Security, OpenAPI)	Implementado
`model`	Entidades JPA, enums e value objects	Implementado
`repository`	Acesso ao banco via Spring Data JPA	Implementado
`service`	Regras de negócio, orquestração, BCrypt	Pendente
`dto`	Objetos de entrada/saída da API (request/response)	Pendente
`controller`	Endpoints REST, mapeamento de rotas, @Valid	Pendente
`exception`	Exceções customizadas + handler global HTTP	Pendente

3 Camada de Domínio — Model

3.1 Entidade User

A entidade `User` é o coração do domínio. Ela representa um usuário do sistema e é mapeada diretamente para a tabela `users` no banco de dados.

SRC/MAIN/JAVA/BR/COM/FIAP/RESTAURANTE/MODEL/USER.JAVA

```
@Entity @Table(name = "users") @Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class User { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
@NotBlank @Size(max = 100) private String nome; @NotBlank @Email @Size(max = 150)
@Column(unique = true, nullable = false) private String email; @NotBlank @Size(min = 3, max = 50)
@Column(unique = true, nullable = false) private String login; @NotBlank private String senha; // sempre
armazenada como hash BCrypt @Column(nullable = false) private LocalDateTime dataUltimaAlteracao;
@NotNull @Enumerated(EnumType.STRING) @Column(nullable = false) private UserType tipo; @NotNull
@Valid @Embedded private Address endereco; @PrePersist public void prePersist() { dataUltimaAlteracao
= LocalDateTime.now(); } @PreUpdate public void preUpdate() { dataUltimaAlteracao =
LocalDateTime.now(); } }
```

Explicação das anotações usadas em User

Anotação	Camada	O que faz
`@Entity`	JPA	Marca a classe como entidade gerenciada pelo Hibernate
`@Table(name="users")`	JPA	Define o nome exato da tabela no banco (sem essa anotação usaria "user", palavra reservada em SQL)
`@Id`	JPA	Define o campo como chave primária
`@GeneratedValue(IDENTITY)`	JPA	Delega a geração do ID para o banco (BIGSERIAL/auto-increment)
`@Column(unique, nullable)`	JPA	Refina o mapeamento da coluna: unicidade e NOT NULL no DDL
`@Enumerated(STRING)`	JPA	Persiste o enum como texto ("OWNER"/"CUSTOMER") em vez de número ordinal

Anotação	Camada	O que faz
`@Embedded`	JPA	Instrui o Hibernate a incorporar os campos de Address na mesma tabela
`@Valid`	Bean Validation	Cascateia a validação para dentro do objeto Address embutido
`@PrePersist / @PreUpdate`	JPA Lifecycle	Callbacks executados antes de INSERT e UPDATE — atualiza dataUltimaAlteracao automaticamente
`@NotBlank`	Bean Validation	Campo não pode ser null, vazio ou só espaços
`@Email`	Bean Validation	Valida formato de e-mail
`@Size(min, max)`	Bean Validation	Controla tamanho mínimo e máximo da String

3 Camada de Domínio — Model (cont.)

3.2 Value Object Address (@Embeddable)

`Address` não é uma entidade independente — é um **value object** embutido dentro de `User`. Isso significa que não possui tabela própria: seus campos são colunas da tabela `users`.

> 💡 **@Embeddable vs @Entity:** Uma entidade tem identidade própria (id) e tabela própria. Um embeddable não tem id — ele vive dentro de outra entidade. É ideal para dados que não fazem sentido isolados, como endereços.

SRC/MAIN/JAVA/BR/COM/FIAP/RESTAURANTE/MODEL/ADDRESS.JAVA

```
@Embeddable @Builder @NoArgsConstructor @AllArgsConstructor public class Address { @NotBlank private String logradouro; @NotBlank private String numero; private String complemento; // nullable — campo opcional @NotBlank private String bairro; @NotBlank private String cidade; @NotBlank private String estado; @NotBlank @Size(max = 9) @Pattern(regexp = "\\d{5}-?\\d{3}") // aceita "12345-678" ou "12345678" private String cep; }
```

Como os campos de Address ficam no banco

> ✅ Como Address é `@Embeddable`, o Hibernate mapeia cada campo diretamente como coluna de `users`. Não há JOIN — todos os dados de um usuário ficam em uma única linha da tabela.

Campo em Address	Coluna na tabela users	Nullable?
`logradouro`	`logradouro`	Não
`numero`	`numero`	Não
`complemento`	`complemento`	Sim
`bairro`	`bairro`	Não
`cidade`	`cidade`	Não
`estado`	`estado`	Não
`cep`	`cep`	Não

3.3 Enum UserType

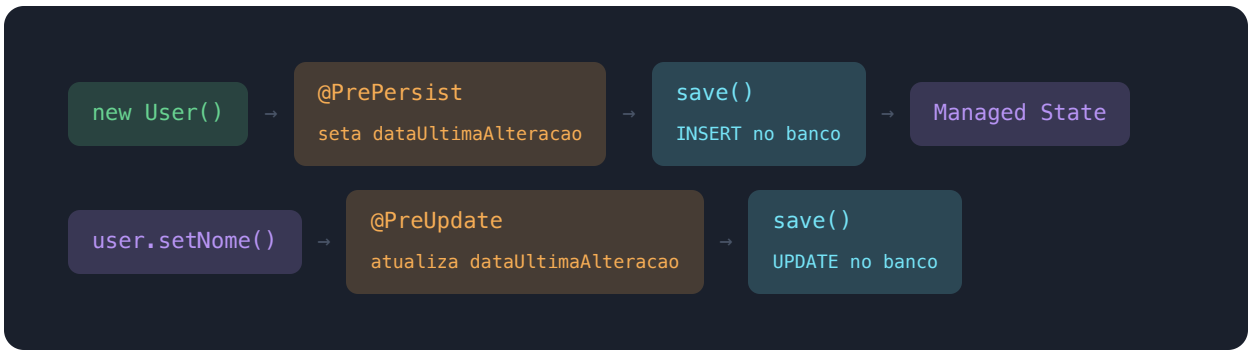
O enum diferencia os dois perfis de usuário do sistema. É persistido como texto graças ao `@Enumerated(EnumType.STRING)` na entidade User.

```
public enum UserType { OWNER, // Dono de Restaurante CUSTOMER // Cliente }
```

> ⚠️ **Por que STRING e não ORDINAL?** Com `EnumType.ORDINAL` o valor 0 ou 1 é salvo. Se um novo valor for inserido no meio do enum no futuro, todos os registros existentes ficam com valores errados. Com `EnumType.STRING`, o texto "OWNER"/"CUSTOMER" é salvo — resistente a mudanças de ordem.

3.4 Ciclo de vida JPA da entidade User

O diagrama abaixo mostra quando os callbacks `@PrePersist` e `@PreUpdate` são disparados:



4 Repositório — UserRepository

O repositório é a camada responsável pelo acesso ao banco de dados. Com Spring Data JPA, não é necessário escrever SQL manualmente para operações simples — basta declarar uma interface que estende `JpaRepository` e o Spring gera a implementação em tempo de execução.

SRC/MAIN/JAVA/BR/COM/FIAP/RESTAURANTE/REPOSITORY/USERREPOSITORY.JAVA

```
public interface UserRepository extends JpaRepository<User, Long> { Optional<User> findByEmail(String email); Optional<User> findByLogin(String login); boolean existsByEmail(String email); boolean existsByLogin(String login); }
```

O que JpaRepository já fornece gratuitamente

Método herdado	SQL gerado	Uso previsto
`save(user)`	INSERT ou UPDATE	Cadastro e atualização de usuário
`findById(id)`	SELECT ... WHERE id = ?	Buscar usuário por ID
`deleteById(id)`	DELETE ... WHERE id = ?	Excluir usuário
`existsById(id)`	SELECT count(*) WHERE id = ?	Verificar existência antes de operar
`findAll()`	SELECT * FROM users	Listar todos (uso interno/admin)

Queries derivadas por nome de método

O Spring Data JPA interpreta o nome do método e **gera o SQL automaticamente**. Não há implementação manual.

Método declarado	SQL gerado pelo Spring	Retorno
`findByEmail(email)`	`SELECT * FROM users WHERE email = ?`	`Optional<User>` — vazio se não encontrado
`findByLogin(login)`	`SELECT * FROM users WHERE login = ?`	`Optional<User>` — usado na validação de login

Método declarado	SQL gerado pelo Spring	Retorno
`existsByEmail(email)`	`SELECT count(*) > 0 WHERE email = ?`	`boolean` — mais eficiente que findBy para checar unicidade
`existsByLogin(login)`	`SELECT count(*) > 0 WHERE login = ?`	`boolean` — verificação de duplicidade no cadastro

💡 **Por que Optional e não User diretamente?** Retornar `Optional<User>` força o código chamador a tratar explicitamente o caso de "não encontrado", eliminando NullPointerException e tornando o código mais seguro e legível. O Service deve chamar `.orElseThrow(() -> new UserNotFoundException(...))`.

5 Migrations com Flyway

O Flyway é responsável por criar e evoluir o schema do banco de dados de forma controlada e versionada. Em vez de deixar o Hibernate criar as tabelas automaticamente (ddl-auto: create), usamos scripts SQL explícitos — o que garante controle total sobre o schema e evita surpresas em produção.

Como funciona o Flyway

```
1. Aplicação sobe → Flyway escaneia classpath:db/migration/
2. Lê todos os arquivos no padrão V{versão}__{descrição}.sql
3. Consulta a tabela interna flyway_schema_history no banco
4. Executa apenas os scripts ainda não aplicados (novos)
5. Registra cada execução com checksum na tabela de histórico
6. Aplicação continua a inicialização normalmente
```

Convenção de nomenclatura dos arquivos

```
V1__create_users.sql ← V + versão + __ (dois underscores) + descrição V2__add_index_email.sql
V3__add_column_phone.sql
```

> ⚠️ **Scripts Flyway jamais devem ser alterados após serem executados.** O Flyway grava o checksum de cada arquivo. Se o conteúdo mudar, ele lança erro na próxima subida. Para corrigir algo, sempre crie um novo arquivo V2__, V3__...

Script V1 — Criação da tabela users

SRC/MAIN/RESOURCES/DB/MIGRATION/V1__CREATE_USERS.SQL

```
-- V1: Criação da tabela de usuários CREATE TABLE users ( id BIGSERIAL PRIMARY KEY, nome
VARCHAR(100) NOT NULL, email VARCHAR(150) NOT NULL UNIQUE, login VARCHAR(50) NOT NULL
UNIQUE, senha VARCHAR(255) NOT NULL, -- hash BCrypt (60 chars) data_ultima_alteracao TIMESTAMP
NOT NULL, tipo VARCHAR(20) NOT NULL, -- OWNER ou CUSTOMER -- Campos do Address
(@Embedded) logradouro VARCHAR(200) NOT NULL, numero VARCHAR(20) NOT NULL, complemento
```

```
VARCHAR(100), -- nullable bairro VARCHAR(100) NOT NULL, cidade VARCHAR(100) NOT NULL, estado
VARCHAR(2) NOT NULL, -- sigla UF (SP, RJ...) cep VARCHAR(9) NOT NULL -- formato: XXXXX-XXX );
```

Pontos importantes do script

- `BIGSERIAL` é o tipo PostgreSQL para Long auto-increment (equivale a BIGINT + sequência automática)
- `senha VARCHAR(255)` — o hash BCrypt tem sempre 60 caracteres, mas 255 dá margem para outros algoritmos futuros
- `tipo VARCHAR(20)` — armazena o texto do enum ("OWNER" ou "CUSTOMER")
- `estado VARCHAR(2)` — apenas a sigla da UF (SP, RJ, MG...)
- Os campos de endereço fazem parte da mesma tabela (sem JOIN) por causa do `@Embedded`

6 Configurações da Aplicação

6.1 application.yml — Perfil padrão (produção/Docker)

SRC/MAIN/RESOURCES/APPLICATION.YML

```
spring: datasource: url:
jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:restaurante_db} username:
${DB_USER:postgres} password: ${DB_PASSWORD:postgres} jpa: hibernate: ddl-auto: validate # Flyway
cuida do schema — Hibernate só valida properties: hibernate: dialect:
org.hibernate.dialect.PostgreSQLDialect flyway: enabled: true locations: classpath:db/migration baseline-
on-migrate: true server: port: ${SERVER_PORT:8080}
```

Variáveis de ambiente e seus defaults

Variável	Default (local)	Valor no Docker	Finalidade
`DB_HOST`	localhost	postgres	Host do PostgreSQL
`DB_PORT`	5432	5432	Porta do banco
`DB_NAME`	restaurante_db	restaurante_db	Nome do banco
`DB_USER`	postgres	postgres	Usuário do banco
`DB_PASSWORD`	postgres	postgres	Senha do banco
`SERVER_PORT`	8080	8080	Porta HTTP da app

💡 **Por que `ddl-auto: validate`?** Com essa configuração, o Hibernate não cria nem altera tabelas — ele apenas verifica se o schema atual do banco é compatível com as entidades JPA. Qualquer divergência causa erro na inicialização. Quem cria o schema é exclusivamente o Flyway.

6.2 application-test.yml — Perfil de testes (H2 in-memory)

SRC/MAIN/RESOURCES/APPLICATION-TEST.YML

```
spring: datasource: url: jdbc:h2:mem:restaurante_test;DB_CLOSE_DELAY=-1 driver-class-name:
org.h2.Driver jpa: hibernate: ddl-auto: create-drop # cria schema ao subir e destrói ao encerrar properties:
```

```
hibernate: dialect: org.hibernate.dialect.H2Dialect flyway: enabled: false # H2 usa ddl-auto no perfil de teste
h2: console: enabled: true path: /h2-console
```

O perfil `test` é ativado com a anotação `@ActiveProfiles("test")` nos testes:

```
@SpringBootTest @ActiveProfiles("test") class RestauranteUsuariosApplicationTests { @Test void
contextLoads() { } // apenas verifica se o contexto Spring sobe sem erros }
```

7 Segurança — SecurityConfig

A configuração de segurança tem dois objetivos nesta fase do projeto: **(1)** disponibilizar o `BCryptPasswordEncoder` como bean para criptografar senhas, e **(2)** desabilitar a autenticação HTTP padrão do Spring Security para que os endpoints fiquem acessíveis.

SRC/MAIN/JAVA/BR/COM/FIAP/RESTAURANTE/CONFIG/SECURITYCONFIG.JAVA

```
@Configuration @EnableWebSecurity public class SecurityConfig { @Bean public PasswordEncoder
passwordEncoder() { return new BCryptPasswordEncoder(); } @Bean public SecurityFilterChain
securityFilterChain(HttpSecurity http) throws Exception { http .csrf(AbstractHttpConfigurer::disable)
.authorizeHttpRequests(auth -> auth .anyRequest().permitAll() ); return http.build(); } }
```

Por que isso é necessário?

Ao incluir `spring-boot-starter-security` no projeto, o Spring Security automaticamente protege **todos** os endpoints com autenticação HTTP Basic. Como o projeto implementa autenticação própria via `POST /usuarios/login`, a proteção padrão é desativada intencionalmente.

Configuração	Efeito	Motivação
`BCryptPasswordEncoder` `@Bean`	Disponibiliza o encoder no container Spring	O Service pode injetar e usar para hash de senhas
`.csrf(disable)`	Desabilita proteção CSRF	APIs REST stateless não usam cookies de sessão, tornando CSRF irrelevante
`.anyRequest().permitAll()`	Libera todos os endpoints sem autenticação HTTP	Autenticação é tratada na camada de negócio via POST /login

Como o BCrypt será usado pelo Service (a implementar)

```
@Service public class UserService { private final PasswordEncoder passwordEncoder; // injetado do
SecurityConfig // Cadastro — criptografar antes de salvar
user.setSenha(passwordEncoder.encode(request.getSenha())); // Login — comparar hash armazenado
com senha informada passwordEncoder.matches(senhaInformada, user.getSenha()); }
```

⚠️ **BCrypt é unidirecional.** Não é possível "descriptografar" uma senha BCrypt. Para validar o login, usa-se `passwordEncoder.matches(raw, hash)` — que recalcula e compara o hash, sem nunca expor a senha original.

8 Swagger / OpenAPI

A documentação interativa da API é gerada automaticamente pelo **Springdoc OpenAPI 2.5.0**. Com essa biblioteca, o Swagger UI é disponibilizado sem nenhuma configuração adicional — basta ter os controllers anotados.

SRC/MAIN/JAVA/BR/COM/FIAP/RESTAURANTE/CONFIG/OPENAPICONFIG.JAVA

```
@Configuration public class OpenApiConfig { @Bean public OpenAPI customOpenAPI() { return new OpenAPI() .info(new Info() .title("Restaurante — Gestão de Usuários") .version("1.0") .description("API para cadastro e gerenciamento de usuários...")); } }
```

Endpoints disponibilizados automaticamente

URL	Descrição
`http://localhost:8080/swagger-ui.html`	Interface gráfica Swagger UI — para testar os endpoints no browser
`http://localhost:8080/v3/api-docs`	Especificação OpenAPI em formato JSON — pode ser importado no Postman

Anotações que a Pessoa 4 usará nos controllers

Anotação	Onde usar	O que documenta
`@Operation(summary = "...")`	Método do controller	Título curto do endpoint no Swagger
`@ApiResponse(responseCode="200")`	Método do controller	Documenta os possíveis retornos HTTP
`@Tag(name = "Usuários")`	Classe controller	Agrupa endpoints na UI do Swagger
`@Schema(description="...")`	Campos do DTO	Descreve campos no schema do payload

✅ A URL **http://localhost:8080/swagger-ui.html** está configurada no `application.yml` via `springdoc.swagger-ui.path`. Após os controllers serem implementados, todos os endpoints

aparecerão automaticamente na interface.

9 Infraestrutura Docker

9.1 Dockerfile — Build multi-stage

O Dockerfile usa **multi-stage build**: o primeiro estágio compila o código com Maven e JDK, o segundo estágio copia apenas o JAR gerado para uma imagem mínima com JRE. Isso resulta em uma imagem final significativamente menor.

Stage 1 — Build

- Imagem base: `maven:3.9.6-eclipse-temurin-17`
- Copia o `pom.xml` e baixa dependências (cache de camada Docker)
- Copia o `src/` e executa `mvn package -DskipTests`
- Gera o JAR em `/app/target/*.jar`

Stage 2 — Runtime

- Imagem base: `eclipse-temurin:17-jre-alpine` (mínima)
- Cria usuário não-root `appuser` por segurança
- Copia o JAR do Stage 1
- Expõe porta 8080
- Executa com `java -jar app.jar`

9.2 docker-compose.yml

Define dois serviços: o banco de dados PostgreSQL e a aplicação Spring Boot.

Serviço	Imagem	Porta	Detalhe importante
`postgres`	`postgres:16-alpine`	5432:5432	Healthcheck com `pg_isready` — a app só sobe após o banco estar pronto
`app`	Build local (Dockerfile)	8080:8080	`depends_on: postgres: condition: service_healthy` — evita falhas de conexão na inicialização

Fluxo de inicialização

```
docker compose up --build
→ Docker constrói a imagem da app (Stage 1: Maven build → Stage 2: JRE)
```

```
→ Sobe o container postgres e aguarda healthcheck (pg_isready)
→ Após postgres healthy, sobe o container app
→ Spring Boot inicia → Flyway executa V1__create_users.sql
→ Hibernate valida schema com ddl-auto: validate
→ API disponível em http://localhost:8080
```

Comandos úteis

```
# Subir tudo (build + banco + app) docker compose up --build # Subir em background docker compose up --build -d # Parar e remover containers docker compose down # Parar e remover containers + volume do banco (reset total) docker compose down -v # Ver logs da aplicação docker compose logs -f app
```

10 Dependências — pom.xml

Dependência	Escopo	Finalidade
`spring-boot-starter-web`	compile	Servidor HTTP embutido (Tomcat), MVC, serialização JSON (Jackson)
`spring-boot-starter-data-jpa`	compile	Hibernate ORM + Spring Data JPA (repositories, @Entity, @Transactional)
`spring-boot-starter-validation`	compile	Jakarta Bean Validation — @NotBlank, @Email, @Size, @Valid
`spring-boot-starter-security`	compile	BCryptPasswordEncoder para hash de senhas
`lombok`	optional	Gera getters/setters/construtores em tempo de compilação — não vai para o JAR final
`spring-boot-devtools`	runtime optional	Restart automático em desenvolvimento — não impacta produção
`postgresql`	runtime	Driver JDBC do PostgreSQL — necessário em tempo de execução
`h2`	runtime ⚠	Banco in-memory para testes — deveria ter escopo `test` para não entrar no JAR de produção
`flyway-core`	compile	Motor central do Flyway para execução das migrations
`flyway-database-postgresql`	compile	Suporte específico do Flyway para PostgreSQL (necessário no Flyway 10+)
`springdoc-openapi-starter-webmvc-ui`	compile	Gera Swagger UI e json de especificação OpenAPI 3 automaticamente
`spring-boot-starter-test`	test	JUnit 5, Mockito, AssertJ, MockMvc — suite completa de testes

Dependência	Escopo	Finalidade
`spring-security-test`	**test**	Suporte a testes com contexto Spring Security (@WithMockUser, etc.)

11 O que falta implementar

O projeto está na fase de setup e domínio. As camadas de aplicação (service, controller, DTOs, exception handler) ainda precisam ser criadas. Abaixo estão todas as tarefas pendentes organizadas por responsável.

Pessoa 3 — Service Layer

#	O que criar	Detalhes
1	Cadastro de usuário	Verificar unicidade de email/login com `existsByEmail/existsByLogin`, hash BCrypt, `save()`
2	Atualização de dados	Buscar por ID, atualizar campos, `save()` — `@PreUpdate` atualiza a data automaticamente
3	Troca de senha	Validar senha atual com `passwordEncoder.matches()`, hash da nova, persistir
4	Validação de login	`findByLogin()`, `passwordEncoder.matches()`, retornar sucesso/falha
5	Exclusão de usuário	`findById()` + `deleteById()`, lançar `UserNotFoundException` se não existir
6	Exceções customizadas	`UserNotFoundException`, `DuplicateEmailException`, `InvalidCredentialsException`

Pessoa 4 — Controllers, DTOs e Exception Handler

#	O que criar	Detalhes
1	DTOs de Request	`CreateUserRequest`, `UpdateUserRequest`, `ChangePasswordRequest`, `LoginRequest`
2	DTOs de Response	`UserResponse` — nunca expor o campo `senha`
3	UserController	POST /usuarios, GET /usuarios/{id}, PUT /usuarios/{id}, DELETE /usuarios/{id}, POST /usuarios/login, PATCH /usuarios/{id}/senha

#	O que criar	Detalhes
4	GlobalExceptionHandler	`@ControllerAdvice` — mapear exceções para HTTP 400, 404, 409, 500
5	Anotações Swagger	`@Operation`, `@ApiResponse`, `@Tag` nos endpoints

Pessoa 5 — Testes e Documentação

#	O que criar	Detalhes
1	Testes unitários do UserService	JUnit 5 + Mockito — cobrir casos de sucesso, duplicidade, senha inválida
2	Collection Postman	Todos os endpoints com body de exemplo, exportar como JSON
3	Validação em ambiente Docker	Subir com `docker compose up` e testar todos os cenários

Endpoints previstos na API

Método	Rota	Descrição	Status
POST	/usuarios	Cadastrar novo usuário	Pendente
GET	/usuarios/{id}	Buscar usuário por ID	Pendente
PUT	/usuarios/{id}	Atualizar dados do usuário	Pendente
DELETE	/usuarios/{id}	Excluir usuário	Pendente
POST	/usuarios/login	Validar login (autenticação)	Pendente
PATCH	/usuarios/{id}/senha	Trocar senha	Pendente

⚠️ **Problema identificado:** O H2 está declarado com escopo `runtime` no pom.xml, o que o inclui no JAR de produção. Alterar para `test` resolve isso. Além disso, o `application-test.yml` está em `src/main/resources` e deveria estar em `src/test/resources` para não ser empacotado junto com o artefato final.